Practical Number 1

Aim/Objective

- Solving 2 water jug Problem: Write a program to solve the 2-water jug problem by manually providing moves from the user.
- Write a program to implement Missionaries and Cannibals problem under the strict constraints.

Theory

Problems such as Two Water Jug problem and Missionaries and Cannibals were discussed and implemented in the first practical of Artificial Intelligence lab. These two problems are based on the idea that a problem is broken into different states and moves between them. This concept is known as State Space Search Problem.

The Two Water Jug problem consists of 2 jugs of fixed capacity. This problem's goal is to find the desired quantity of water in one of the jugs through performing the allowed actions of fill, empty and transfer the water from one jug to another. This problem helps us in understanding how states change through the moves that are performed provided that every move must be valid.

The Missionaries and Cannibals problem consists of 3 missionaries and 3 cannibals in which it is required for everyone to cross the bank of the river using a boat under strict constraints. The boat can carry only 2 people at once. An important constraint that is placed is that the numbers of missionaries must never be outnumbered by the number of cannibals on either of the banks or else the missionaries will be eaten by the cannibals. Also, the boat cannot cross itself with no people onboard.

Algorithm/Methodology

1) Two Water Jug Problem

1) **Step 1**

   Start the program

2) **Step 2**

   Read the capacity of the Jug X from the user(capX).

3) **Step 3**

   Read the capacity of the Jug Y from the user(capY).

4) **Step 4**

   Read the desired amount of data to be measured from the user(target).

5) **Step 5**

   In both jugs, initialise the current amount of water present in both

   i.e. currX=0, currY=0

6) Step 6

Define the following operations:

1. **Fill Jug X** – Fill the jug X to its maximum capacity.
2. **Fill Jug Y** – Fill the jug Y to its maximum capacity.
3. **Empty Jug X** – Empty all the water from jug X.
4. **Empty Jug Y** – Empty all the water from jug Y.
5. **Pour from X to Y** – Transfer water from the jug X to the jug Y until X is empty or Y is full.
6. **Pour from Y to X** – Transfer water from the jug Y to the jug X until Y is empty or X is full.

7)Step 7

Repeat the following steps until the target amount is achieved in either of the jugs:

- Display the current state of jug X and jug Y.
- Display the list of possible operations.

- Ask the user to select an operation.
- Perform the selected operation.

8) Step 8

Check the goal condition:

currX == target

or

currY == target

9) Step 9

If the goal condition is satisfied, display the success message along with the final state of both jugs.

10) Step 10

Finish the program.

2) Missionaries and Cannibals Problem

1) **Step 1**
   Initialize start = (3, 3, 0) and goal = (0, 0, 1).
2) **Step 2**
   Initialize a queue Q and insert (start, empty_path) into it.
3) **Step 3**
   Initialize an empty set as visited.
4) **Step 4**
   While Q is not empty, remove (m, c, boat) and its path from the front of the queue.
5) **Step 5**
   If (m, c, boat) == goal, print path and terminate.
6) **Step 6**
   If (m, c, boat) is in visited, continue to the next iteration.
7) **Step 7**
   For each possible move (dm, dc) in {(1,0),(2,0),(0,1),(0,2),(1,1)}, generate a new state based on boat position.

8) **Step 8**

   If $0 \leq new_m \leq 3$ and $0 \leq new_c \leq 3$ and the state is valid, insert the new state and updated path into Q.

9) **Step 9**

   Add (m, c, boat) to visited and repeat until the goal is reached.

Dataset Description

1) Two Water Jug Problem

Source

User input during program execution.

Attributes

capX: Represents the maximum capacity of Jug X.

capY: Represents the maximum capacity of Jug Y.

target: Indicates the desired amount of water to be measured.

currX: Stores the current quantity of water present in Jug X during execution.

currY: Stores the current quantity of water present in Jug Y during execution.

Size

The dataset size is very small and fixed.

It consists of three input values and two state variables.

Data is processed iteratively rather than stored.

2) Missionaries and Cannibals Problem

Source

Program-generated states during execution

Attributes

m: Number of missionaries on the left bank.

c: Number of cannibals on the left bank.

boat: Indicates boat position (0 = left bank, 1 = right bank).

start: Initial state of the problem (3, 3, 0).

goal: Target state (0, 0, 1).

path: Sequence of states leading from the initial state to the goal.

visited: Stores already explored states to avoid repetition.

Size

Finite and small state space.

Maximum possible states: 32 (4 × 4 × 2).

Suitable for BFS traversal.

Implementation

A) Two Water Jug Problem

```
# Two Water Jug Problem using user-driven approach


# Reading capacities of both jugs and the target amount

capX = int(input("Enter the capacity of Jug X: "))

capY = int(input("Enter the capacity of Jug Y: "))

target = int(input("Enter the target amount: "))


# Initializing current water levels in both jugs

currX = 0

currY = 0


# Function to fill Jug X completely

def fillX():

    global currX
```

```
    currX = capX
    print("Jug X is filled")

# Function to fill Jug Y completely
def fillY():
    global currY
    currY = capY
    print("Jug Y is filled")

# Function to empty Jug X
def emptyX():
    global currX
    currX = 0
    print("Jug X is emptied")

# Function to empty Jug Y
def emptyY():
    global currY
    currY = 0
    print("Jug Y is emptied")

# Function to pour water from Jug X to Jug Y
def pourFromX():
    global currX, currY
    space = capY - currY
    pour = min(currX, space)
    currX -= pour
    currY += pour
```

```
    print(f"Poured {pour} units from Jug X to Jug Y")

# Function to pour water from Jug Y to Jug X
def pourFromY():
    global currX, currY
    space = capX - currX
    pour = min(currY, space)
    currY -= pour
    currX += pour
    print(f"Poured {pour} units from Jug Y to Jug X")

# Main function to solve the Two Water Jug Problem
def TwoWaterJugProblem():
    global currX, currY

    # Loop continues until target amount is present in any one jug
    while not (currX == target or currY == target):

        # Displaying the current state of both jugs
        print(f"\nCurrent state: Jug X = {currX}, Jug Y = {currY}")

        # Displaying possible operations
        choice = int(input(
            "Choose an operation:\n"
            "1. Fill Jug X\n"
            "2. Fill Jug Y\n"
            "3. Empty Jug X\n"
            "4. Empty Jug Y\n"
```

```
            "5. Pour from X to Y\n"
            "6. Pour from Y to X\n"
            "Enter choice: "
        ))

        # Performing the selected operation
        if choice == 1:
            fillX()
        elif choice == 2:
            fillY()
        elif choice == 3:
            emptyX()
        elif choice == 4:
            emptyY()
        elif choice == 5:
            pourFromX()
        elif choice == 6:
            pourFromY()
        else:
            print("Invalid choice. Please select a valid operation.")

    # Target state reached
    print(f"\nTarget achieved! Final state: Jug X = {currX}, Jug Y = {currY}")

# Calling the main function
TwoWaterJugProblem()
```

B) Missionaries and Cannibals Problem

```
from collections import deque

# Function to check whether a state is valid
# A state is valid if missionaries are not outnumbered by cannibals
def is_valid(m, c):
    return (m == 0 or m >= c) and (3 - m == 0 or 3 - m >= 3 - c)

# Function to solve the Missionaries and Cannibals problem
def miss_cann():

    # Initial state: 3 missionaries, 3 cannibals, boat on left side (0)
    start = (3, 3, 0)

    # Goal state: 0 missionaries, 0 cannibals, boat on right side (1)
    goal = (0, 0, 1)

    # Queue for BFS storing current state and the path taken
    queue = deque([(start, [])])

    # Set to keep track of visited states
    visited = set()

    # BFS loop
    while queue:
        (m, c, boat), path = queue.popleft()

        # If goal state is reached
        if (m, c, boat) == goal:
            print("Solution Found:")
            for step in path + [(m, c, boat)]:
                print(step)
            return

        # Skip already visited states
        if (m, c, boat) in visited:
            continue

        visited.add((m, c, boat))
```

```
        # Possible moves (missionaries, cannibals)
        for dm, dc in [(1, 0), (2, 0), (0, 1), (0, 2), (1, 1)]:

            # Boat on left side
            if boat == 0:
                new_m = m - dm
                new_c = c - dc
            else:
                # Boat on right side
                new_m = m + dm
                new_c = c + dc

            # Change boat position
            new_boat = 1 - boat

            # Check bounds and validity of new state
            if 0 <= new_m <= 3 and 0 <= new_c <= 3:
                if is_valid(new_m, new_c):
                    queue.append(((new_m, new_c, new_boat),
                                  path + [(m, c, boat)]))

# Function call
miss_cann()
```

Output/Result

A) Two Water Jug Problem

```
[1]
2m
TwoWaterJugProblem()

Enter the capacity of X: 3
Enter the capacity of Y: 5
Enter the target amount: 4

Current state: X=0, Y=0
Enter choice: 1.Fill X, 2.Fill Y, 3.Empty X, 4.Empty Y, 5.Pour X→Y, 6.Pour Y→X: 2
Y is filled

Current state: X=0, Y=5
Enter choice: 1.Fill X, 2.Fill Y, 3.Empty X, 4.Empty Y, 5.Pour X→Y, 6.Pour Y→X: 6
Poured 3 from Y to X

Current state: X=3, Y=2
Enter choice: 1.Fill X, 2.Fill Y, 3.Empty X, 4.Empty Y, 5.Pour X→Y, 6.Pour Y→X: 3
X is emptied

Current state: X=0, Y=2
Enter choice: 1.Fill X, 2.Fill Y, 3.Empty X, 4.Empty Y, 5.Pour X→Y, 6.Pour Y→X: 6
Poured 2 from Y to X

Current state: X=2, Y=0
Enter choice: 1.Fill X, 2.Fill Y, 3.Empty X, 4.Empty Y, 5.Pour X→Y, 6.Pour Y→X: 2
Y is filled

Current state: X=2, Y=5
Enter choice: 1.Fill X, 2.Fill Y, 3.Empty X, 4.Empty Y, 5.Pour X→Y, 6.Pour Y→X: 6
Poured 1 from Y to X

Target achieved! Final state: X=3, Y=4
```

B) Missionaries and Cannibals Problem

```
[1]
0s
miss_cann()

Solution Found:
(3, 3, 0)
(3, 1, 1)
(3, 2, 0)
(3, 0, 1)
(3, 1, 0)
(1, 1, 1)
(2, 2, 0)
(0, 2, 1)
(0, 3, 0)
(0, 1, 1)
(1, 1, 0)
(0, 0, 1)
```

Result Analysis

A) Two Water Jug Problem

The program, is correct and verified as the correct amount of desired quantity is obtained as a result and displayed successfully. All functions such as filling, emptying and transferring the water in sequence have happened, which is a proof that state transitions and conditions are handled correctly.

B) Missionaries and Cannibals Problem

The program generated valid and correct states during the execution. At every step the constraints were obeyed. The output follows the sequence well and transfers all missionaries and cannibals to the right bank. Thus, this problem was solved successfully .

Conclusion

This practical covers two main problems i.e. Two Water Jug problem and Missionaries and Cannibals problem here which were solved by keeping the constraints in mind and by problem solving ways in AI.